# A Real-Time Hand Sign Recognition Pipeline for Gaming

Arham Muse Hassan
Arham.musehassan@student.uliege.be

Hiba Qouiqa
Hiba.Qouiqa@student.uliege.be

Madanassono Pouezi Paring
madanassono.paring@student.uliege.be

## Abstract

*We demonstrate the use, as is, of an existing generic machine learning model as a sub-module in order to achieve a classification task in a specific domain. The goal involved assessing the complementary needs with regard to the generic sub-module, which is the MediaPipe Hand Landmarker task model [1], developing the specialized part of the model, and assembling the pieces into a pipeline. In this work, the choice is to recognize in images the different hand signs a player might perform during a Naruto game [2]. Furthermore, for the purpose of a realistic use of our model, we make the classification on live real-time videos. Performance evaluation on the test set exhibits a satisfactory global score of 96% correct classifications, while some of the Naruto hand signs are obviously less successfully recognized than others. Codebase is available https://github.com/Akram3112/Real-Time-Hand-Sign-Recognition-Pipeline-for-Gaming.*

## 1. Introduction

Virtual reality is a maturing technology, and with the recent rapid advances in computer vision, it is now possible to assemble well-known state-of-the-art models to construct remarkable pipelines. Games have taken incredible leaps forward; titles such as *Call of Duty*, *Mario Kart*, *God of War*, and *Grand Theft Auto V* stand as testaments to how far the industry has come. The gaming community is ready and willing to pay for new immersive experiences, yet augmented reality gaming remains surprisingly underexplored. Among the best-known attempts at physical interaction are the Nintendo Wii and Xbox , whose setups allow a "kind" of augmented reality, such as the Wii steering wheel (see Figure 2), but turning a plastic wheel left and right is far less stimulating than it once felt back in 2008.

Today's games demand strategy, reasoning, and deep engagement. This is precisely the spirit behind *Naruto*, a Japanese anime that first aired in 2002 and became a cultural phenomenon worldwide, including in France and Belgium. Far more than a simple animated series, *Naruto* is built upon profound themes of perseverance, reasoning, empathy, and emotional intelligence, making it one of the most celebrated anime franchises of its generation. It is also renowned for its originality, and in particular for the introduction of hand signs, also known as *mudra* (see Figure 1), which characters perform in precise sequences to unleash powerful techniques.

Figure 1. Example of hand gestures used in *Naruto* [3]. Reference: *http://artpictures.club/autumn-2023.html*

The idea is therefore to design an entirely original game experience powered by augmented reality, in which the

player interacts through *Naruto*-style hand gestures rather than a conventional controller. Rather than relying on the gamepad found in PlayStation 4 and 5 controllers, the goal is to bring the player into the game by having them perform their own hand gestures, recorded in real time by a camera. This approach introduces a new layer of physical and cognitive engagement into a fully original game universe built around the *Naruto* hand sign system. It is worth noting that today's gaming community is no longer limited to young children: it spans all ages, includes highly dedicated enthusiasts, and even professional players who compete at the international level and make a living from it.

Incorporating this augmented reality layer makes the game more immersive and rewarding. If a player wants to execute a technique, they must know the correct sequence of *mudra* and perform it accurately; the faster and more precisely they do so, the faster their character attacks or defends. This adds a compelling layer of complexity to explore, as the player is no longer constrained by the buttons of a gamepad, but only by the dexterity and knowledge of their own hands.

Figure 2. The Nintendo Wii steering wheel, an early attempt at physical game interaction.

The goal of this work is therefore to introduce a pipeline that leverages existing, well-known models to achieve this exciting task. The paper is organized as follows. Section 2 is a brief review of related work. Section 3 provides a brief definition of the task, including the inputs, outputs, and how the pipeline should be interpreted. Section 4 is devoted to the dataset: how the data were collected, why, and some low-level details about their structure. Section 5 describes our pipeline, its organization, and how its components interact. Finally, Section 6 presents the experimental results, including an evaluation of the pipeline, a discussion of its limitations, and qualitative observations.

## 2. Related work

Since the inception of the Naruto game, the automatic classification of hand signs used in the game has gained some attraction within the Machine Learning community.

For example, [5] using a version of YOLO [4] implemented with Tensorflow a model that detects Naruto's hand sign using object detection in images.

[3], on the other hand, adopted a pipeline architecture that consists of two major components: (1) a *Hand Detection module* which uses the MediaPipe Hand Landmarker task model output top determine the hands region and then crops the image to retain that region of interest, and (2) a *Gesture Recognition module* to which the cropped image is fed to for classification by a machine learning model.

Our proposed architecture is similar to the latter, with the difference that we feed to the second block of the pipeline the coordinates of the at most 42 hand landmarks retrieved from the MediaPipe model. Not dealing with all the pixels of images, arguably, our approach is computationally more efficient.

## 3. Problem Statement

As introduced in Section 1, the goal of this work is to design a real-time pipeline capable of recognizing which *mudra* a player is performing. Since the system is intended to be used in a live game environment, real-time inference is not optional but a core requirement of the task.

The second design choice concerns the scope of the classification problem. In the *Naruto* universe, twelve distinct hand signs exist, each with its own specific finger configuration and spatial structure, as illustrated in Figure 1. The task is therefore a 12-class classification problem, where the twelve classes correspond to the signs of the Chinese zodiac, which are the names traditionally associated with the *Naruto* hand signs:

1. Rat,
2. Ox,
3. Tiger,
4. Hare,
5. Dragon,
6. Snake,
7. Horse,
8. Ram,
9. Monkey,
10. Bird,
11. Dog,
12. Boar.

Formally, the task can be defined as follows. The input is a real-time video stream captured by a standard camera, from which individual frames are processed continuously. For each frame, the pipeline must output a predicted class $\hat{y} \in \{1, \ldots, 12\}$ corresponding to the *mudra* being performed, or alternatively signal the absence of a recognized hand sign.

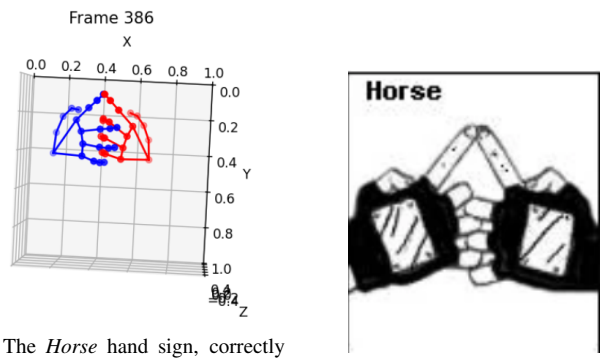
A key design decision concerns the representation of the hand used as input to the classifier. Rather than relying on raw 2D image patches, we leverage 3D hand landmark coordinates, which capture the true geometric structure of a gesture independently of where the hand appears in the frame. This makes the representation significantly more robust to translation and scale variations than standard 2D image-based approaches, while preserving all the spatial in-

formation that is discriminative for hand sign classification.

## 4. Dataset

The data used in this work was collected specifically for this project. This choice was motivated by two reasons: first, to confront the real challenges of data collection in practice, and second, to ensure that the data is as representative as possible of the actual conditions in which the pipeline will be used, namely a player performing hand signs in front of a standard webcam.

### 4.1. Collection Protocol




(a) The *Horse* hand sign, correctly detected by MediaPipe with both hands clearly separated.

(b) Reference: correct *Horse* hand sign.

Figure 3. Illustration of the successful detection for *Horse* hand sign.

The goal being a twelve-class classification problem, data was collected for each of the twelve *mudra*. Each participant was shown a reference image of the target hand sign, such as the one shown in Figure 3, and was asked to reproduce the gesture for approximately 10 seconds in front of their computer's built-in webcam, operating at 30 frames per second. This yields approximately 300 frames per gesture per participant. Participants were also encouraged to slowly shift their hand position sideways during the recording, in order to introduce viewpoint variability and capture different angles of the same gesture. Furthermore, the first two seconds of each recording were discarded, as the participant may still be transitioning into the correct position during that initial phase, introducing unwanted noise into the data.

In total, 11 participants were recruited, 7 males and 4 females, recording in different locations and under different lighting conditions to introduce variability into the dataset. This results in a total of $11 \times 12 \times 300 = 39{,}600$ raw frames before any processing.

### 4.2. Feature Extraction

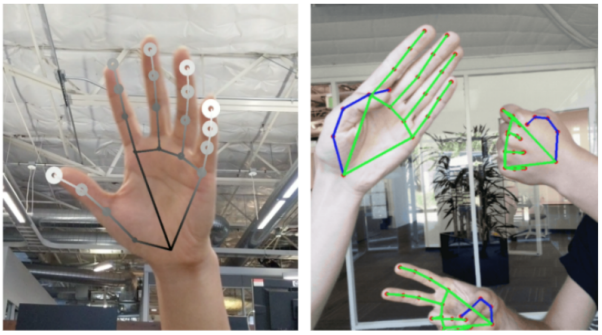


Figure 4. MediaPipe Hands keypoint detection [6].

Rather than working directly with raw video frames, a feature extraction step was applied to convert the recordings into a compact and meaningful representation. A standard webcam image of resolution $1280 \times 720$ pixels produces thousands of raw features per frame, and feeding such high-dimensional input directly into a classifier would be both computationally wasteful and statistically problematic given the modest size of the dataset.

Instead, MediaPipe Hands [6] was used as a feature extractor. As illustrated in Figure 4, MediaPipe Hands detects and tracks the 21 keypoints of each hand in real time, outputting their 3D world coordinates in metric space. Applied to both hands, this reduces each frame to a $2 \times 21 \times 3 = 126$-dimensional feature vector[1], independently of the original image resolution.

This abstraction brings several important benefits. By operating on 3D landmark coordinates rather than raw pixels, the representation becomes inherently robust to variations in lighting, skin color, and camera distance. It is worth noting, however, that since the features are hand coordinates in metric space, a subtle bias may exist between male and female participants, as hand size differs on average between the two groups. This effect is considered minor and is noted here for completeness.

### 4.3. Annotation

The annotation process was fully automated at recording time. A script processed each video frame, extracted the 3D hand landmark coordinates using MediaPipe, and stored the resulting feature vectors in a structured folder hierarchy organized by class label. This eliminates any manual annotation effort and ensures perfect label consistency across the dataset.

[1] 2 hands , 21 landmark , $3D$ coordinates

### 4.4. Dataset Statistics and Splits

The dataset was collected following a strict and uniform protocol, ensuring an identical number of frames per class and thus no class imbalance. The dataset was split following a participant-wise strategy: all frames from a given participant belong exclusively to one split including the mirrored frames, preventing any data leakage between sets. The split was performed as follows: $\approx 15\%$ for testing (11,558 frames), also $\approx 15\%$ for validation (11,586 frames), and the rest approximately $\approx 70\%$ for training (39,230 frames).

One notable issue encountered during feature extraction concerns frames in which MediaPipe fails to detect one of the two hands. This occurs when the two hands are in close proximity and one occludes the other, preventing the detector from localizing it. The resulting missing landmark ratios per class are reported in Table 1.

Table 1. Missing hand landmark ratios per class across the full dataset.

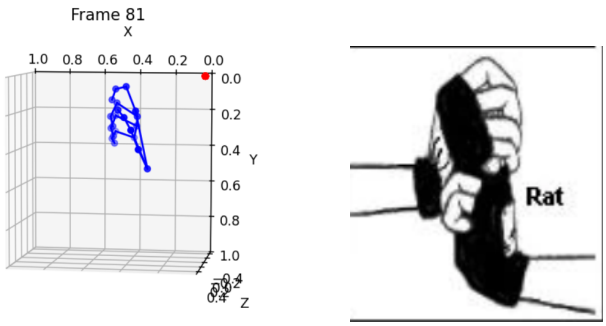
| Class | Left (%) | Right (%) | Both missing (%) |
|---|---|---|---|
| Bird | 0.0 | 0.0 | 0.0 |
| Boar | 0.0 | 0.2 | 0.0 |
| Dog | 0.0 | 6.6 | 0.0 |
| Dragon | 0.0 | 0.0 | 0.0 |
| Hare | 0.0 | 0.0 | 0.0 |
| Horse | 0.0 | 0.0 | 0.0 |
| Monkey | 0.0 | 9.6 | 0.0 |
| Ox | 0.0 | 4.0 | 0.0 |
| Ram | 0.0 | 0.0 | 0.0 |
| Rat | 0.0 | 0.0 | 0.0 |
| Snake | 0.0 | 0.1 | 0.0 |
| Tiger | 0.0 | 5.7 | 0.0 |

### 4.5. Data Augmentation

The decision of whether to apply data augmentation to handle missing hand landmarks was carefully examined. Initially, two approaches were considered: mirroring the detected hand to replace the missing one, and estimating the missing hand using interpolation from nearby frames. These methods were first rejected because the missing detections are not caused by noise in the data, but by limitations of the MediaPipe extractor itself. This point is illustrated in Figures 3a and 5. The Horse sign (Figure 3a) presents two clearly separated hands, resulting in perfect detection. The Rat sign, by contrast, brings the two hands into close contact, causing MediaPipe to lose track of one of them, as shown in Figure 5.

However, due to the limited size of the dataset, mirror augmentation was later introduced to increase the diversity of the training data. Unlike the previous strategies, mirroring does not attempt to reconstruct missing hands, but instead generates additional valid samples by flipping the hand configuration. Experimental results showed that this significantly improves performance. For example, test accuracy increased from $67.5\%$ to $83.7\%$ with a Decision Tree model, and from $81.9\%$ to $93.7\%$ with an MLP model.

So based on this experiment mirror augmentation was kept as part of the final preprocessing pipeline.




(a) MediaPipe output: only one hand detected.

(b) Reference: correct *Rat* hand sign.

Figure 5. Illustration of the detection failure for the *Rat* sign, where the proximity of the two hands causes MediaPipe to miss one of them.

The preferred strategy is therefore to preserve the data as collected and rely on the classifier's ability to learn from the naturally occurring missing patterns. Since the inter-class distances in the feature space are sufficiently large in practice, as demonstrated experimentally in Section 6, this approach does not harm classification performance.

## 5. Pipeline

The pipeline used here could therefore be any machine learning algorithm, as the input consists of only 126 features, nothing compared to the thousands, if not millions, of features that would have been required had raw images been used directly. Furthermore, the modest size of the dataset makes this a problem where generalization from limited data is critical.

The pipeline is presented in Figure 6. It is composed of two stages: a feature extractor based on the MediaPipe Hand Landmarker task model, and a classifier that operates on the resulting 126-dimensional feature vector.

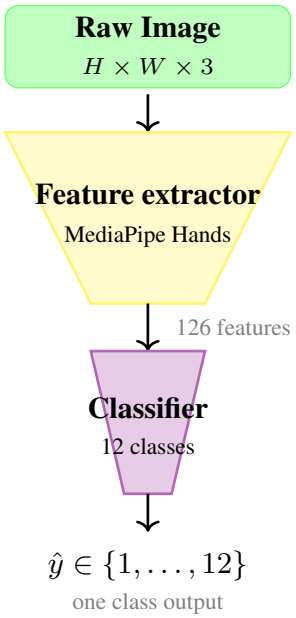



Figure 6. Pipeline overview. Raw image frames are passed through MediaPipe Hands (feature extractor), reducing the input to 126 features, which are then classified into one of the twelve *mudra* classes.

Two classifiers were evaluated: a Decision Tree and a Multilayer Perceptron (MLP) implemented from scratch. The results of both models are presented in Section 6.

### 5.1. Decision Tree

For the Decision Tree, all hyperparameters were left at their default values, in order to serve as an untuned baseline and allow a fair comparison with the optimized model.

### 5.2. Multilayer Perceptron

The MLP was chosen as a second classifier in order to learn the non-linear relationships between the extracted features. We refer to this architecture as **MudraMLP**. The MLP architecture and training procedure are described in detail below.

#### 5.2.1. Architecture

The network consists of two hidden layers. The input layer receives the 126-dimensional feature vector. The first hidden layer projects the input to 256 neurons, followed by Batch Normalisation, a ReLU activation function, and a dropout layer with a dropout rate of 40%. The second hidden layer reduces the representation to 128 neurons, followed again by Batch Normalisation, ReLU activation, and a dropout rate of 25%.

The final output layer is a linear layer mapping to 12 classes. A softmax function is applied during inference to obtain class probabilities.

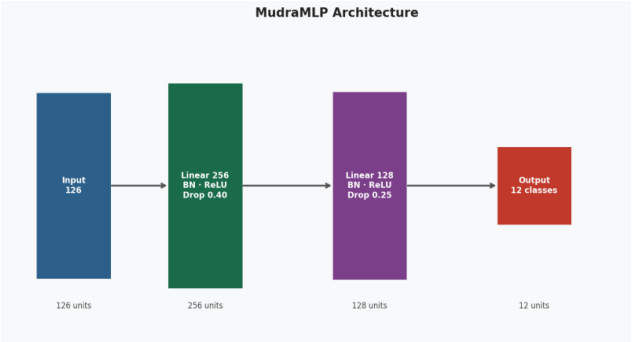



Figure 7. Architecture of the proposed MudraMLP model.

#### 5.2.2. Training Setup

Training was performed using the Adam optimiser with a learning rate of $5 \times 10^{-4}$ and a weight decay of $1 \times 10^{-4}$. A batch size of 128 was used throughout training.

To address residual class imbalance in the dataset, a class-weighted cross-entropy loss function was employed. In addition, a `ReduceLROnPlateau` scheduler was used to automatically reduce the learning rate by a factor of two after five consecutive validation epochs without improvement.

Early stopping with a patience value of 15 epochs was also applied. The model checkpoint corresponding to the lowest validation loss was saved as the final model.

## 6. Experiment

### 6.1. Implementation details

To better understand the impact of the preprocessing choices, several experiments were conducted using different preprocessing configurations and classifiers. Two models were evaluated: a Decision Tree used as a simple baseline model, and an MLP used as the main optimised model. Different preprocessing strategies were tested, including no augmentation, mirror augmentation, mirror augmentation with scale normalisation, and mirror augmentation with frame deduplication. All experiments were performed using the same participant-based dataset split (70% training, 15% validation, and 15% testing) to ensure fair comparison between the different configurations.

Scale normalisation hurts performance $-4.9\%$ because our raw world coordinates already carry absolute positional information that helps the model, and normalising removes it. However the biggest gain was adding mirror augmentation, it doubles training diversity by flipping hand configurations horizontally and swapping left/right roles producing more physically valid sign variants at zero recording cost.

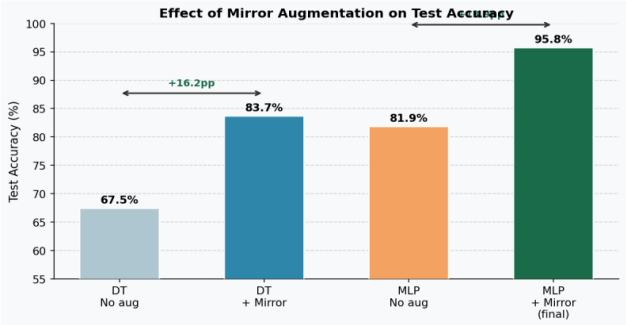



Figure 8. Test accuracy before and after mirror augmentation for both classifiers. The MLP gains $+13.9\%$; the Decision Tree gains $+16.2\%$.

## 6.2. Evaluation

Performance evaluation is based on the top-1 accuracy of the Naruto hand sign prediction.

## 6.3. Results and Analysis

Figure 9 shows a confusion matrix for the Test set.

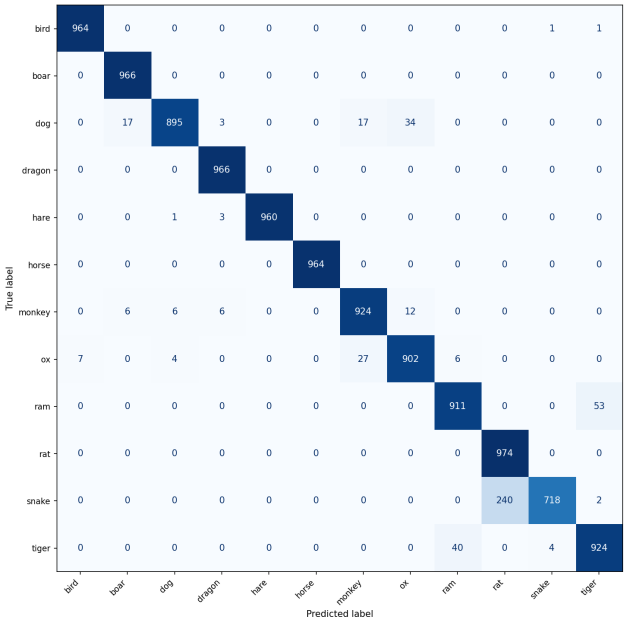



Figure 9. MLP: confusion matrix of the test set. The *Horse* hand sign is perfectly classified while the False Positive rate of *Rat* is high for *Snake* (Figure 11b) tending, one in four, to be taken for it.

| LABELS | COUNT | TP | | TN | | FN | | FP | | TOTAL |
|---|---|---|---|---|---|---|---|---|---|---|
| | # | # | % | # | % | # | % | # | % | # |
| bird | 966 | 964 | 99.8% | 10585 | 91.6% | 2 | 0.02% | 7 | 0.72% | 11558 |
| boar | 966 | 966 | 100.0% | 10569 | 91.4% | 0 | 0.00% | 23 | 2.33% | 11558 |
| dog | 966 | 895 | 92.7% | 10581 | 91.5% | 71 | 0.61% | 11 | 1.21% | 11558 |
| dragon | 966 | 966 | 100.0% | 10580 | 91.5% | 0 | 0.00% | 12 | 1.23% | 11558 |
| hare | 964 | 960 | 99.6% | 10594 | 91.7% | 4 | 0.03% | 0 | 0.00% | 11558 |
| horse | 964 | 964 | 100.0% | 10594 | 91.7% | 0 | 0.00% | 0 | 0.00% | 11558 |
| monkey | 954 | 924 | 96.9% | 10560 | 91.4% | 30 | 0.26% | 44 | 4.55% | 11558 |
| ox | 946 | 902 | 95.3% | 10566 | 91.4% | 44 | 0.38% | 46 | 4.85% | 11558 |
| ram | 964 | 911 | 94.5% | 10548 | 91.3% | 53 | 0.46% | 46 | 4.81% | 11558 |
| rat | 974 | 974 | 100.0% | 10344 | 89.5% | 0 | 0.00% | 240 | 19.77% | 11558 |
| snake | 960 | 718 | 74.8% | 10593 | 91.7% | 242 | 2.09% | 5 | 0.69% | 11558 |
| tiger | 968 | 924 | 95.5% | 10534 | 91.1% | 44 | 0.38% | 56 | 5.71% | 11558 |

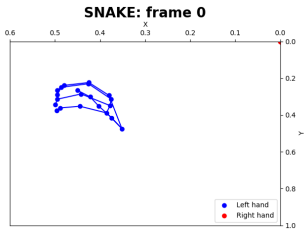
Figure 10. The true positive (TP), true negative (TN), false negative (FN) and false positive (FP) scores for each of the Naruto hand signs.

The *Horse* hand sign (Figure 3b) is undoubtedly the best detected hand sign, achieving a score of 100% for both accuracy, precision and recall; a good result was indeed anticipated for that sign, as its geometric configuration allows a perfect detection of all the hand landmarks.

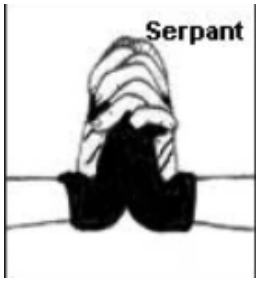
On the other hand, while the *Boar*, *Dragon* and *Rat* hand signs are well detected leading to a 100 % recall score, other signs tend to be confused with them. On this ground, the infamous hand sign is the *Snake* (Figure 11b), which is taken for the *Rat* (Figure 11d) 25 % of the time.

The true positive (TP), true negative (TN), false negative (FN) and false positive (FP) scores for each of the Naruto hand signs are available in Figure 10.

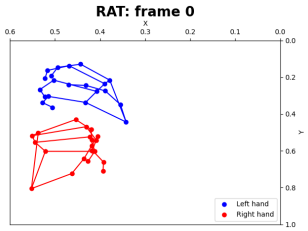
**Qualitative Results.** Figures 11a and 11c are the plots of the hand landmarks detected by the Mediapipe module for both signs a same person performed. It can be seen that the shape of the *Snake* hand landmarks resembles a lot to a part (about the half) of the shape of that of the *Rat* hand sign. In that performance, the *Snake* sign is lacking all of its right hand features (but only one landmark considered to pertaining to the right hand). Given that such an observation is not an isolated case, our assumption is that it explains the high rate of the misclassification of the *Snake* signs taken not scarcely for a *Rat* sign.



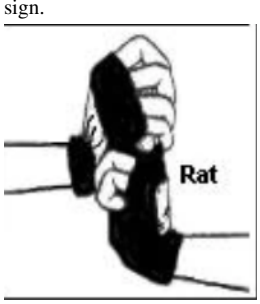

(a) Detected hand landmarks from a performed *Snake* sign.




(b) Reference: *Snake* hand sign.




(c) Detected hand landmarks from a performed *Rat* sign.




(d) Reference: *Rat* hand sign.

Figure 11. Juxtaposition of *Snake* and *Rat* signs.

**Decision Tree results.** Compared to the MLP, the Decision Tree confusion matrix exhibits the same global trend as Figure 12 highlights it, yet the scores are worse.

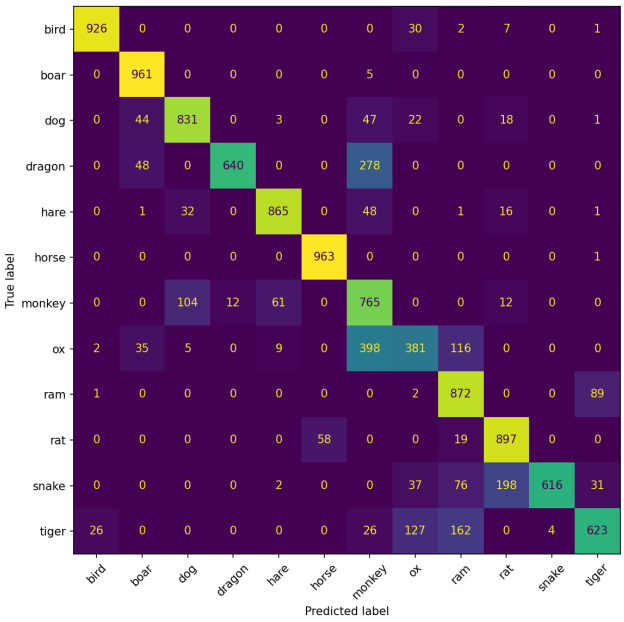



Figure 12. Decision Tree: confusion matrix of the test set.

## 7. Conclusion

We have developed a Machine Learning model based on the MediaPipe Hand Landmarker task, which is a generic model for detecting the hand landmarks. The complementary needs to make it suited to our specific purpose, the classification of the Naruto hand signs, led us to exploring many machine learning approaches. We have presented the results obtained with a Decision tree classifier and multilayer perceptron classifier, the latter on which the main focus was and from which our selected best model is. Hovewer the results of our selected model are overall satisfactory, some of the Naruto hand signs are less accurately detected than others, especially the *Snake* which tends not scarcely to be taken for the *Rat*. Future efforts are thus anticipated to focus on better discriminating those two Naruto hand signs.

## References

[1] Mediapipe hand landmarks detection guide. https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker. 1

[2] Naruto official website. https://naruto-official.com, accessed: May 05, 2026. 1

[3] ben tiki. Naruto hand sign recognition, 2023. https://github.com/ben-tiki/naruto-handsign-recognition. GitHub repository. Accessed: : February 21, 2026. 1, 2

[4] Zheng Ge, Songtao Liu, Feng Wang, Zeming Li, and Jian Sun. Yolox: Exceeding yolo series in 2021, 2021. 2

[5] Kazuhito00. Naruto-handsigndetection, 2021. https://github.com/Kazuhito00/NARUTO-HandSignDetection. GitHub repository. Accessed: February 21, 2026. 2

[6] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang, and Matthias Grundmann. Mediapipe hands: On-device real-time hand tracking, 2020. https://arxiv.org/abs/2006.10214. 3